

Formation CDA

Angular - Routing

Andrej RACIC - NUMERICA

Sommaire

1 — Router, RouterLink, RouterOutlet, ActivatedRoute

2 — Guards, routes enfants et lazy loading

Router, RouterLink, RouterOutlet et ActivatedRoute

Objectifs

- Comprendre ce qu'est le routing côté client dans une SPA.
- Configurer les routes dans app.routes.ts.
- Utiliser RouterOutlet pour afficher les composants selon la route.
- Naviguer avec RouterLink dans les templates.
- Naviguer programmatiquement avec le service Router.
- Lire les paramètres de route et les query params avec ActivatedRoute.

1. Le routing dans une SPA (Single Page Application) — pourquoi et comment

Dans une application web classique (sans framework), chaque URL correspond à une page HTML différente sur le serveur — chaque clic sur un lien provoque un rechargement complet. Dans une SPA Angular, il n'y a qu'une seule page HTML (index.html) : le routing est géré entièrement en JavaScript, sans rechargement. L'URL change dans la barre d'adresse, mais c'est Angular qui remplace le contenu affiché.

Analogie : le routing comme une télécommande de TV

Dans une application classique, changer de chaîne = éteindre la télé et en allumer une autre. Dans Angular, c'est comme changer de chaîne avec une télécommande : l'écran (la page) reste le même, seul le contenu change. L'URL est le numéro de chaîne — Angular sait quel composant afficher pour chaque URL.

Architecture de routing de l'application bibliothèque

```
# URLs de l'application bibliothèque
http://localhost:4200/           -> Accueil
http://localhost:4200/livres     -> ListeLivres
http://localhost:4200/livres/64a1b -> DetailLivre (id = 64a1b)
http://localhost:4200/emprunts   -> ListeEmprunts
http://localhost:4200/profil     -> Profil
http://localhost:4200/login      -> Login
```

```
http://localhost:4200/admin/livres -> AdminLivres (route enfant)
```

2. Configurer les routes — app.routes.ts

Les routes Angular sont définies dans app.routes.ts sous forme d'un tableau de Route. Chaque objet Route associe un chemin URL (path) à un composant (component). Angular lit ce tableau et affiche le composant correspondant quand l'URL correspond.

```
// src/app/app.routes.ts
import { Routes } from '@angular/router';
import { Accueil }      from './pages/accueil/accueil';
import { ListeLivres }  from './pages/liste-livres/liste-livres';
import { DetailLivre }  from './pages/detail-livre/detail-livre';
import { Profil }       from './pages/profil/profil';
import { Login }        from './pages/login/login';

export const routes: Routes = [
  // Route exacte — path: '' correspond à http://localhost:4200/
  { path: '',          component: Accueil },

  // Route avec chemin fixe
  { path: 'livres',    component: ListeLivres },

  // Route avec paramètre dynamique — :id est une variable
  // http://localhost:4200/livres/64a1b2c3 -> id = '64a1b2c3'
  { path: 'livres/:id', component: DetailLivre },

  { path: 'profil',    component: Profil },
  { path: 'login',     component: Login },

  // Redirection — path: '**' capture tout ce qui ne correspond pas
  { path: '**',        redirectTo: '' },
  // Toute URL inconnue -> redirection vers la page d'accueil
];
```

Enregistrer les routes dans app.config.ts

```
// src/app/app.config.ts
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes)],
};
```

```
};
```

3. RouterOutlet — où afficher les composants

RouterOutlet est une directive Angular qui marque l'endroit dans le template où Angular doit afficher le composant correspondant à la route active. C'est le 'trou' dans lequel Angular insère le bon composant selon l'URL.

```
// app.ts — importer RouterOutlet
import { RouterOutlet, RouterLink } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink, MatToolbarModule,
    MatSidenavModule],
  templateUrl: './app.html',
})
export class App { }

<!-- app.html -->
<mat-toolbar>
  <span>Bibliothèque CDA</span>
  <span class='spacer'></span>
  <!-- RouterLink pour naviguer sans rechargement de page -->
  <a mat-button routerLink='/livres'>Catalogue</a>
  <a mat-button routerLink='/emprunts'>Mes emprunts</a>
  <a mat-button routerLink='/profil'>Profil</a>
</mat-toolbar>

<mat-sidenav-container>
  <mat-sidenav-content>
    <!-- RouterOutlet = emplacement où Angular affiche le composant
    actif -->
    <!-- Quand l'URL = /livres, Angular affiche ListeLivres ici -->
    <!-- Quand l'URL = /profil, Angular affiche Profil ici -->
    <router-outlet></router-outlet>
  </mat-sidenav-content>
</mat-sidenav-container>
```

4. RouterLink — naviguer depuis le template

RouterLink est une directive de navigation déclarative. Elle génère un lien exploitable par le navigateur et laisse le Router Angular intercepter la navigation interne afin de changer de vue sans rechargement complet de la page.

```
<!-- Syntaxe simple — chemin fixe -->
<a routerLink='/livres'>Catalogue</a>
<a routerLink='/profil'>Mon profil</a>

<!-- Syntaxe tableau — pour construire des chemins dynamiques -->
<!-- [routerLink]='["/livres", livre._id]' -> /livres/64a1b2c3 -->
<a [routerLink]='["/livres", livre._id]'>Voir ce livre</a>
<a [routerLink]='["/admin", "livres"]'>Admin livres</a>

<!-- routerLinkActive — ajoute une classe CSS quand la route est active -->
<a routerLink='/livres'
  routerLinkActive='lien-actif'
  [routerLinkActiveOptions]='{ exact: true }'>
  Catalogue
</a>
<!-- exact: true = la classe est ajoutée SEULEMENT si l'URL correspond exactement -->
<!-- sans exact, /livres/64a1b serait aussi considéré comme actif pour /livres -->

<!-- Avec Angular Material — même principe sur mat-list-item -->
<a mat-list-item routerLink='/livres' routerLinkActive='active'>
  Catalogue
</a>
```

5. Navigation programmatique — le service Router

Parfois on veut naviguer depuis le code TypeScript (après un login réussi, après la création d'un livre, après une erreur...). Le service Router fournit la méthode `navigate()` pour cela.

```
// login.ts
import { Component, inject } from '@angular/core';
import { Router } from '@angular/router';
import { AuthService } from '../../services/auth.service';

@Component({ /* ... */ })
export class Login {
  private router = inject(Router);
```

```

private authService = inject(AuthService);

onLogin(email: string, motDePasse: string): void {
  this.authService.login({ email, mot_de_passe: motDePasse
}).subscribe({
  next: (reponse) => {
    // Stocker le token
    localStorage.setItem('token', reponse.data.token);

    // Naviguer vers la page d'accueil après le login
    this.router.navigate(['/']);
    // Ou vers une page spécifique :
    // this.router.navigate(['/livres']);
  },
  error: (err) => {
    console.error('Erreur de connexion :', err);
  },
});
}

// navigate() avec options
allerAuLivre(id: string): void {
  // Équivalent de [routerLink]='["/livres", id]'
  this.router.navigate(['/livres', id]);
}

// navigateByUrl() — avec l'URL complète
retourAccueil(): void {
  this.router.navigateByUrl('/');
}
}

```

6. ActivatedRoute — lire les paramètres de la route

ActivatedRoute est un service injecté automatiquement dans chaque composant activé par le Router. Il donne accès aux informations de la route active : paramètres de chemin (:id), query params (?page=2&q=hugo), fragment (#section)...

Analogie : ActivatedRoute comme le bon de livraison

Quand un livreur (le Router) vous apporte un colis (active un composant), il vous remet aussi un bon de livraison (ActivatedRoute) qui détaille ce que vous avez commandé : quel produit (le path), pour quelle quantité (les params), avec quelles options (les query params).

Lire un paramètre de route — :id

```
// detail-livre.ts
// Route configurée : { path: 'livres/:id', component: DetailLivre }
// URL active       : http://localhost:4200/livres/64a1b2c3d4e5

import { Component, OnInit, inject } from '@angular/core';
import { ActivatedRoute }              from '@angular/router';
import { LivreService }                from
'../../services/livre.service';
import { Livre }                      from '../../models/livre.model';

@Component({
  imports:      [],
  templateUrl: './detail-livre.html',
})
export class DetailLivre implements OnInit {
  private route      = inject(ActivatedRoute);
  private livreService = inject(LivreService);

  livre:      Livre | null = null;
  chargement: boolean      = true;

  ngOnInit(): void {
    // snapshot : valeur de l'id au moment de l'activation du composant
    // Toujours une string – convertir si nécessaire
    const id = this.route.snapshot.paramMap.get('id');

    if (id) {
      this.livreService.getLivreById(id).subscribe({
        next: (reponse) => {
          this.livre      = reponse.data;
          this.chargement = false;
        },
        error: () => { this.chargement = false; }
      });
    }
  }
}
```

Lire les query params — ?page=2&q=hugo

```
// liste-livres.ts
// URL : http://localhost:4200/livres?page=2&q=hugo

import { ActivatedRoute, Router } from '@angular/router';
```



```

export class ListeLivres implements OnInit {
  private route = inject(ActivatedRoute);
  private router = inject(Router);

  page:      number = 1;
  recherche: string = '';

  ngOnInit(): void {
    // Lire les query params depuis le snapshot
    const params = this.route.snapshot.queryParamMap;
    this.page     = parseInt(params.get('page') || '1');
    this.recherche = params.get('q') || '';

    this.chargerLivres();
  }

  // Mettre à jour les query params dans l'URL sans rechargement
  changerPage(nouvellePage: number): void {
    this.page = nouvellePage;
    this.router.navigate([], {
      relativeTo:      this.route,
      queryParams:      { page: nouvellePage, q: this.recherche ||
null },
      queryParamsHandling: 'merge', // conserver les autres query params
    });
    this.chargerLivres();
  }

  chargerLivres(): void {
    // Utiliser this.page et this.recherche pour l'appel API
  }
}

```

snapshot vs Observable params

`route.snapshot.paramMap` est une photo des paramètres au moment de l'activation. Elle convient si le composant est recréé pour chaque URL. Si Angular réutilise la même instance pendant qu'un paramètre change (par exemple `/livres/1` puis `/livres/2`), abonnez-vous à `route.paramMap` ou transformez-le avec RxJS afin de réagir aux nouvelles valeurs.

Tableau récapitulatif — Router vs RouterLink vs ActivatedRoute

	Rôle	Utilisé dans	Exemple
RouterOutlet	Emplacement d'affichage	Template	<router-outlet>
RouterLink	Navigation déclarative	Template	routerLink='/livres'
Router	Navigation programmatique	TypeScript	this.router.navigate(['/'])
ActivatedRoute	Lire les paramètres	TypeScript	this.route.snapshot.paramMap

Exercices

Continuez sur le projet bibliotheque-front. Générez les composants pages avec ng g c pages/nom-page.

Exercice 1 — Configurer les routes de l'application

Dans app.routes.ts, configurez les routes suivantes :

- path: " -> Accueil (créez ce composant)
- path: 'livres' -> ListeLivres
- path: 'livres/:id' -> DetailLivre (créez ce composant)
- path: 'emprunts' -> ListeEmprunts
- path: 'login' -> Login (créez ce composant)
- path: '**' -> redirection vers "

Dans app.html, ajoutez <router-outlet></router-outlet> dans le mat-sidenav-content. Testez que la navigation fonctionne en changeant l'URL manuellement dans le navigateur.

Exercice 2 — Ajouter la navigation avec RouterLink et routerLinkActive

Dans le MatSidenav de App, remplacez les liens existants par des liens RouterLink vers /livres, /emprunts et /login. Ajoutez routerLinkActive='active' sur chaque lien et définissez la classe .active en CSS (fond coloré, texte blanc). Vérifiez que le lien de la page active est bien mis en évidence lors de la navigation.

Exercice 3 — Créer la page détail d'un livre avec ActivatedRoute

Dans DetailLivre, injectez ActivatedRoute et LivreService. Dans ngOnInit, lisez l'id avec route.snapshot.paramMap.get('id') et chargez le livre correspondant avec getLivreById(id). Affichez le titre, l'auteur, l'ISBN, l'année et la disponibilité avec Angular Material (MatCard). Affichez un MatProgressSpinner pendant le chargement et un message d'erreur si le livre n'est pas trouvé. Dans CarteLivre, faites en sorte que le titre soit un RouterLink vers /livres/:id.

Exercice 4 — Navigation programmatique après création

Dans `ListeLivres`, ajoutez une méthode `onCreerLivre()` qui simule la création d'un livre (objet hardcodé) en appelant `livreService.creerLivre()` puis navigue vers `/livres` après le succès. Injectez `Router` et utilisez `this.router.navigate(['/livres'])` dans le callback `next`. Testez le flux : cliquez sur 'Créer' ; après le succès, la navigation revient sur la liste sans rechargement complet de la page.

Exercice 5 — Pagination avec les query params

Dans `ListeLivres`, implémentez la pagination avec les query params. Lisez `page` et `q` depuis `route.snapshot.queryParamMap` dans `ngOnInit`. Ajoutez deux boutons 'Page précédente' et 'Page suivante' qui appellent `changerPage()`. Cette méthode met à jour l'URL avec `router.navigate([], { queryParams: { page } })` et recharge les livres. Vérifiez que l'URL dans le navigateur reflète bien la page courante.

Guards, routes enfants et lazy loading

Objectifs

- Comprendre ce qu'est un Guard et pourquoi en avoir besoin.
- Créer un Guard CanActivate pour protéger des routes.
- Configurer des routes enfants pour une section admin.
- Comprendre le lazy loading et ses bénéfices.
- Implémenter le lazy loading sur une section admin / un groupe de routes admin.

1. localStorage — persister des données dans le navigateur

localStorage est une API du navigateur qui permet de stocker des données sous forme de paires clé/valeur. Les données persistent entre les rechargements de page et les sessions du navigateur — contrairement aux variables JavaScript qui sont effacées quand on quitte la page. Dans Angular, on l'utilise pour stocker le token JWT après le login.

Analogie : localStorage comme un tiroir personnel

localStorage est comme un tiroir dans votre bureau : vous y rangez des choses (setItem) et vous pouvez les récupérer plus tard (getItem), même après avoir quitté et rouvert votre bureau. Chaque site web a son propre tiroir — les données d'un site ne sont pas accessibles par un autre.

```
// Stocker une valeur simple (string)
localStorage.setItem('token',
  'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...');
localStorage.setItem('theme', 'dark');

// Lire une valeur — retourne null si la clé n'existe pas
const token = localStorage.getItem('token');
console.log(token); // 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...' ou null

// Supprimer une entrée
localStorage.removeItem('token');

// Vider tout le localStorage
```

```
localStorage.clear();

// localStorage ne stocke que des strings
// Pour stocker un objet, il faut le convertir en JSON
const user = { _id: '64alb', nom: 'Alice', role: 'admin' };

// JSON.stringify() : objet -> string JSON
localStorage.setItem('user', JSON.stringify(user));

// JSON.parse() : string JSON -> objet
const userStr = localStorage.getItem('user');
if (userStr) {
  const userRecupere = JSON.parse(userStr);
  console.log(userRecupere.nom); // 'Alice'
  console.log(userRecupere.role); // 'admin'
}
```

 localStorage — ne jamais stocker de données sensibles non chiffrées

localStorage est accessible par n'importe quel JavaScript de la page — y compris des scripts malveillants (attaque XSS). Ne jamais stocker de mot de passe ou de données bancaires. Un token JWT est acceptable car il a une durée de vie limitée et ne contient pas de mot de passe.

Dans notre application, on utilisera localStorage pour deux choses :

- **Le token JWT** : stocké après le login avec `localStorage.setItem('token', ...)` et lu dans chaque requête HTTP protégée.
- **L'objet user** : stocké après le login avec `JSON.stringify` pour afficher le nom dans la navbar et vérifier le rôle dans les Guards.

2. Les Guards — protéger les routes

Un Guard est une fonction que le Router Angular exécute avant d'activer une route. Si le Guard retourne `true`, la navigation continue. S'il retourne `false`, la navigation est bloquée. S'il retourne un `UrlTree` (ou une commande de redirection), la navigation courante est annulée et Angular effectue la redirection. C'est le mécanisme qui protège les routes privées : sans Guard, n'importe qui peut accéder à `/profil` ou `/admin` simplement en tapant l'URL.

Analogie : le Guard comme un vigile à l'entrée

Avant d'entrer dans une salle VIP (la route protégée), le vigile (le Guard) vérifie votre badge (le token JWT). Si vous avez un badge valide, vous entrez. Sinon, il vous renvoie à l'accueil (redirection vers /login). La salle VIP elle-même ne sait pas qu'elle est protégée — c'est le vigile qui gère l'accès.

Créer un Guard avec le CLI

```
ng generate guard guards/auth
# Raccourci : ng g guard guards/auth
# Le CLI demande quel type de Guard créer :
# ? Which type of guard would you like to create?
#   -> CanActivate

# Fichier créé : src/app/guards/auth.guard.ts
```

Implémenter le Guard d'authentification

```
// src/app/guards/auth.guard.ts
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';

// CanActivateFn = type d'une fonction Guard moderne (Angular 15+)
// Plus simple que la classe CanActivate de l'ancienne approche
export const authGuard: CanActivateFn = (route, state) => {
  const router = inject(Router);

  // Vérifier si le token existe dans le localStorage
  const token = localStorage.getItem('token');

  if (token) {
    // Token présent -> autoriser la navigation
    return true;
  }

  // Pas de token -> rediriger vers /login
  // router.createUrlTree() crée une URL de redirection
  return router.createUrlTree(['/login'], {
    queryParams: { returnUrl: state.url },
  });

  // Note : on verra par la suite comment vérifier la validité du token
  // JWT
  // Pour l'instant, on vérifie seulement sa présence
};
```

Appliquer le Guard sur les routes

```
// app.routes.ts – appliquer authGuard
import { authGuard } from './guards/auth.guard';

export const routes: Routes = [
  { path: '', component: Accueil },
  { path: 'login', component: Login },
  { path: 'livres', component: ListeLivres },
  { path: 'livres/:id', component: DetailLivres },

  // Routes protégées – authGuard vérifie l'authentification
  { path: 'emprunts', component: ListeEmprunts,
    canActivate: [authGuard] },
  { path: 'profil', component: Profil,
    canActivate: [authGuard] },

  // Route admin – deux guards : connecté ET admin
  { path: 'admin', component: Admin,
    canActivate: [authGuard, adminGuard] },

  { path: '**', redirectTo: '' },
];

// canActivate accepte un tableau – tous les guards doivent retourner
// true
// Si l'un retourne false, la navigation est bloquée
```

Guard admin — vérifier le rôle

```
// src/app/guards/admin.guard.ts
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';

export const adminGuard: CanActivateFn = (_route, state) => {
  const router = inject(Router);
  const userStr = localStorage.getItem('user');

  if (!userStr) return router.createUrlTree(['/login'], { queryParams: {
    returnUrl: state.url } });

  try {
    const user = JSON.parse(userStr) as { role?: string };
    return user.role === 'admin' ? true : router.createUrlTree(['/']);
  } catch {
    return router.createUrlTree(['/login'], { queryParams: { returnUrl:
      state.url } });
  }
}
```

```
};
```

⚠ Les Guards ne remplacent pas la sécurité côté serveur

Un Guard contrôle uniquement la navigation dans l'interface. Le JavaScript du navigateur peut être contourné : l'API doit vérifier le JWT, les rôles et les droits pour chaque opération sensible. Le Guard améliore l'expérience utilisateur, mais ne constitue jamais la sécurité principale.

3. Les routes enfants — structurer les sections

Les routes enfants permettent de définir une hiérarchie d'URLs dans laquelle un composant parent contient un RouterOutlet pour afficher ses enfants. C'est idéal pour une section admin qui a sa propre mise en page (avec un menu admin) et plusieurs sous-pages.

```
// app.routes.ts — routes enfants pour la section admin
import { AdminLayout } from './pages/admin/admin-layout';
import { AdminLivres } from './pages/admin/admin-livres';
import { AdminUsers } from './pages/admin/admin-users';

{ path: 'admin',
  component: AdminLayout, // composant parent avec son propre layout
  canActivate: [authGuard, adminGuard],
  children: [
    // /admin -> redirige vers /admin/livres
    { path: '', redirectTo: 'livres', pathMatch: 'full' },
    // /admin/livres -> affiche AdminLivres dans le RouterOutlet
    { path: 'livres', component: AdminLivres },
    { path: 'users', component: AdminUsers },
  ],
},
```

AdminLayout — le composant parent

```
// admin-layout.ts
import { Component } from '@angular/core';
import { RouterOutlet, RouterLink, RouterLinkActive } from '@angular/router';
import { MatTabsModule } from '@angular/material/tabs';

@Component({
  imports: [RouterOutlet, RouterLink, RouterLinkActive, MatTabsModule],
```



```

template: `
  <h2>Administration</h2>
  <!-- Navigation interne à la section admin -->
  <nav>
    <a routerLink='livres' routerLinkActive='active'>Livres</a>
    <a routerLink='users' routerLinkActive='active'>Utilisateurs</a>
  </nav>
  <!-- RouterOutlet des enfants - AdminLivres ou AdminUsers -->
  <router-outlet></router-outlet>
`
),
}))
export class AdminLayout { }

// Dans les routes enfants, les chemins sont RELATIFS au parent
// routerLink='livres' (sans /) -> /admin/livres
// routerLink='/livres' (avec /) -> /livres (route racine !)

```

pathMatch: 'full' — redirection exacte

Quand on utilise `redirectTo` sur une route enfant vide (`path: ''`), il faut ajouter `pathMatch: 'full'` pour indiquer que la redirection ne s'applique que quand le chemin est exactement vide. Sans ça, Angular appliquerait la redirection à toutes les routes enfants (car toutes commencent par une chaîne vide).

4. Le lazy loading — charger à la demande

Par défaut, Angular charge tous les composants de l'application au démarrage. Pour une grande application, cela ralentit le chargement initial. Le lazy loading permet de ne charger un composant (ou un groupe de routes) que quand l'utilisateur y navigue réellement.

Analogie : le lazy loading comme un buffet à la demande

Dans un buffet normal (eager loading), tous les plats sont servis en même temps dès l'arrivée — même ceux que vous ne mangerez jamais. Avec le lazy loading, on ne vous apporte un plat que quand vous le demandez. Le repas commence plus vite, et vous ne recevez que ce que vous voulez vraiment.

Lazy loading avec loadComponent

```
// app.routes.ts — lazy loading avec loadComponent()
export const routes: Routes = [
  { path: '', component: Accueil }, // eager — chargé au démarrage
  { path: 'login', component: Login }, // eager

  // Lazy loading — chargé seulement quand l'utilisateur navigue vers /livres
  {
    path: 'livres',
    loadComponent: () =>
      import('./pages/liste-livres/liste-livres')
        .then(m => m.ListeLivres),
    // import() = import dynamique JavaScript
    // Retourne une Promesse qui résout le module quand Angular en a besoin
  },

  {
    path: 'livres/:id',
    loadComponent: () =>
      import('./pages/detail-livre/detail-livre')
        .then(m => m.DetailLivre),
  },

  // Lazy loading d'un groupe de routes (section admin)
  {
    path: 'admin',
    canActivate: [authGuard, adminGuard],
    loadChildren: () =>
      import('./pages/admin/admin.routes')
        .then(m => m.adminRoutes),
    // admin.routes.ts contient les routes enfants de la section admin
    // Ce fichier entier n'est chargé que si l'user navigue vers /admin
  },

  { path: '**', redirectTo: '' },
];
```

admin.routes.ts — fichier de routes séparé

```
// src/app/pages/admin/admin.routes.ts
import { Routes } from '@angular/router';

export const adminRoutes: Routes = [
  {
    path: '',
```

```

loadComponent: () =>
  import('./admin-layout').then(m => m.AdminLayout),
children: [
  { path: '', redirectTo: 'livres', pathMatch: 'full' },
  {
    path: 'livres',
    loadComponent: () =>
      import('./admin-livres').then(m => m.AdminLivres),
  },
  {
    path: 'users',
    loadComponent: () =>
      import('./admin-users').then(m => m.AdminUsers),
  },
],
},
];

// Ce fichier entier — et tous les composants qu'il référence —
// ne sont chargés que quand l'utilisateur navigue vers /admin

```

Vérifier le lazy loading dans le navigateur

Ouvrez les DevTools > onglet Network > filtrer 'JS'. Au premier chargement, vous verrez les chunks initiaux. Naviguez vers une route lazy : un nouveau fichier .js apparaît dans le Network — c'est le chunk téléchargé à la demande. La prochaine fois que vous naviguez vers cette route, le chunk est déjà en cache — pas de nouveau téléchargement.

Exercices

Continuez sur le projet bibliotheque-front. Assurez-vous d'avoir un token valide dans localStorage (connectez-vous via POST /auth/login depuis l'API).

Exercice 1 — Créer le Guard d'authentification

Générez ng g guard guards/auth. Implémentez authGuard : lit le token dans localStorage.getItem('token'), retourne true si le token existe, retourne un UrlTree vers /login avec queryParams: { returnUrl: state.url } sinon. Testez en naviguant vers /emprunts sans être connecté (localStorage vide) — vous devez être redirigé vers /login. Connectez-vous (ajoutez un token manuellement dans localStorage via DevTools), rechargez et naviguez vers /emprunts — vous devez y accéder.

Exercice 2 — Protéger les routes emprunts et profil

Dans app.routes.ts, ajoutez canActivate: [authGuard] sur les routes /emprunts et /profil (créez Profil si absent). Testez les trois scénarios dans le navigateur : non connecté -> /emprunts redirige vers /login, connecté (token en localStorage) -> /emprunts accessible, lien direct dans la barre d'adresse vers /profil -> même

comportement.

Exercice 3 — Créer le Guard admin et tester les accès

Générez `ng g guard guards/admin`. Implémentez `adminGuard` : lit l'objet `user` depuis `localStorage.getItem('user')`, le parse avec `JSON.parse()`, vérifie que `user.role === 'admin'`. Si admin : retourne `true`. Si connecté mais pas admin : retourne `router.createUrlTree(['/'])`. Si pas connecté : retourne un `UrlTree` vers `/login` en conservant `state.url` dans `returnUrl`. Créez `Admin` et protégez `/admin` avec `[authGuard, adminGuard]`. Testez avec un user normal (redirigé vers `/`) et un admin (accès accordé).

Exercice 4 — Configurer les routes enfants admin

Créez `AdminLayout` avec son propre template : un titre 'Administration', deux liens `RouterLink` relatifs vers 'livres' et 'users' avec `routerLinkActive`, et un `<router-outlet>`. Créez `AdminLivres` et `AdminUsers` (contenu minimal). Configurez les routes enfants dans `app.routes.ts` avec `children: []`. Vérifiez que `/admin/livres` et `/admin/users` affichent bien le bon composant enfant dans le `RouterOutlet` d'`AdminLayout`.

Exercice 5 — Implémenter le lazy loading

Convertissez les routes `/livres`, `/livres/:id`, `/emprunts` et `/profil` en lazy loading avec `loadComponent`. Ouvrez les DevTools > Network > JS avant de naviguer. Chargez l'application : observez que les chunks de ces pages ne sont PAS chargés.

Naviguez vers `/livres` : un nouveau chunk `.js` apparaît dans le réseau. Naviguez vers `/livres/:id` : un autre chunk. Constatez que la navigation suivante vers `/livres` ne retélécharge pas le chunk (cache).